

PORTADA

# Arquitectura de Software

## **Semana 15 — Arquitecturas Cloud Native, Microservicios (DDD, CQRS, Event Sourcing, API Gateway) y Serverless (FaaS)**

De un datacenter propio a cientos de microservicios en la nube — cierre de la Unidad 4 — con el caso real de Netflix · Universidad de Antioquia · Ingeniería de Sistemas · 2554608 · 2026-2



# Agosto de 2008: tres días sin poder enviar un solo DVD.

Netflix — el incidente que inició la migración más citada de la industria hacia la nube.



## TRANSICIÓN

# De un negocio de DVD por correo a la plataforma de streaming más grande del mundo

En 2008, Netflix todavía enviaba DVDs por correo y operaba sobre una base de datos relacional monolítica, alojada en su propio datacenter. Un incidente de corrupción de esa base de datos detuvo por completo los envíos durante tres días — y encendió una alarma sobre qué tan frágil era depender de una sola pieza de infraestructura centralizada.

**Ese incidente, no una moda tecnológica, es el origen documentado de la migración de Netflix hacia una arquitectura distribuida en la nube — el caso que abre el cierre de la Unidad 4.**

# La migración de Netflix a la nube

|                          |                                                                                                                                                            |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Empresa                  | Netflix, Inc.                                                                                                                                              |
| Incidente disparador     | Corrupción de base de datos, agosto de 2008 — tres días sin poder enviar DVDs a los suscriptores                                                           |
| Decisión                 | Migrar de un datacenter propio, con una base de datos relacional centralizada, a una arquitectura de microservicios distribuidos sobre Amazon Web Services |
| Duración de la migración | Aproximadamente siete años, de 2009 a 2016                                                                                                                 |
| Hito de cierre           | En febrero de 2016, Netflix anunció públicamente el cierre de su último datacenter propio                                                                  |
| Documentación            | Extensamente documentado en el Netflix Technology Blog y en charlas públicas de su entonces arquitecto en la nube, Adrian Cockcroft                        |

CASO · LA TRANSFORMACIÓN

## De un monolito a cientos de microservicios independientes

La migración no fue solo "mover servidores a AWS" — fue una reescritura arquitectónica completa. Netflix descompuso su aplicación monolítica en **cientos de microservicios independientes**, cada uno responsable de una función de negocio concreta (recomendaciones, reproducción, facturación, catálogo), desplegados y escalados de forma autónoma.

**Esta migración es, en la práctica, el caso de estudio más citado de la industria para ilustrar el paso de una arquitectura tradicional a una arquitectura cloud native** — el término formal que veremos a continuación.

CASO · LA CULTURA DE RESILIENCIA

## Chaos Monkey: romper el sistema a propósito

Como parte de esta transformación, en 2011 el equipo de Netflix creó **Chaos Monkey**, una herramienta que apaga aleatoriamente instancias de servidores en producción durante horario laboral, **a propósito**. La idea: si un servicio no puede sobrevivir a que Chaos Monkey lo apague sin avisar, tampoco sobrevivirá a una falla real de infraestructura.

Chaos Monkey se liberó como código abierto en 2012, como parte de un conjunto de herramientas conocido como el "Simian Army" — y es hoy la referencia histórica de lo que la industria llama "ingeniería del caos" (chaos engineering).

TRANSICIÓN

## De la anécdota de Netflix al vocabulario formal

Lo que Netflix construyó tiene nombre propio en la literatura de arquitectura: **Cloud Native**. Cerramos la Unidad 4 formalizando ese término, los patrones de diseño que hacen posible dividir un sistema en microservicios de forma coherente (DDD, CQRS, Event Sourcing), la pieza que los expone al mundo exterior (API Gateway) y el paso final de esta evolución: Serverless.

## CONCEPTO

# ¿Qué es Cloud Native?

Según la definición oficial de la **Cloud Native Computing Foundation (CNCF)**, las tecnologías cloud native permiten construir y ejecutar aplicaciones escalables en entornos modernos y dinámicos como la nube pública, privada o híbrida.

- **Contenedores:** empaquetan cada servicio con sus dependencias, de forma portable entre entornos.
- **Orquestación dinámica:** un sistema (típicamente Kubernetes) decide dónde y cuándo ejecutar cada contenedor, y lo reinicia automáticamente si falla.
- **Microservicios:** la aplicación se organiza como servicios pequeños e independientes, como los que descompuso Netflix (s05).

Cloud native no significa simplemente "correr en la nube" — significa diseñar el sistema para aprovechar la elasticidad y la naturaleza distribuida de la nube desde el principio.

CONCEPTO

# The Twelve-Factor App

En 2011, el equipo de la plataforma en la nube Heroku publicó "**The Twelve-Factor App**", una metodología de doce principios para construir aplicaciones aptas para la nube — hoy una referencia estándar para diseñar servicios cloud native.

| Algunos de los doce factores  | Qué exige                                                                                                     |
|-------------------------------|---------------------------------------------------------------------------------------------------------------|
| Config                        | La configuración se guarda en variables de entorno, nunca en el código.                                       |
| Procesos sin estado           | El servicio no guarda estado local entre solicitudes — cualquier instancia puede atender cualquier solicitud. |
| Disponibilidad/desechabilidad | Cada instancia puede arrancar o detenerse en segundos, sin efectos secundarios.                               |
| Paridad dev/prod              | El entorno de desarrollo debe parecerse lo más posible al de producción.                                      |

CONCEPTO · REPASO

## Microservicios (Semana 8): la base sobre la que se construye todo lo demás

En la Semana 8 vieron el patrón POSA **Microservicios**: servicios pequeños, independientes, cada uno con su propia base de datos, comunicados por red. Cloud native (s08) es, en gran medida, **el entorno de infraestructura que hace viable operar cientos de microservicios a la vez** — como los de Netflix — sin que el costo operativo sea inmanejable.

**Pero dividir un sistema en microservicios plantea una pregunta que el patrón POSA no resuelve por sí solo: ¿dónde exactamente se traza el límite entre un servicio y otro?** Esa pregunta es el punto de partida de Domain-Driven Design.

## CONCEPTO

# Domain-Driven Design (DDD): dónde trazar el límite de un servicio

**Domain-Driven Design (DDD)**, formalizado por Eric Evans en su libro de 2003, es un enfoque de diseño que estructura el software alrededor del dominio del negocio, no de la tecnología. Dos de sus conceptos son la herramienta más usada hoy para decidir la frontera de un microservicio:

- **Bounded Context (contexto delimitado)**: una frontera explícita dentro de la cual un modelo de negocio y su terminología son válidos y consistentes — ej. "Pedido" significa algo distinto en el contexto de Ventas que en el de Logística.
- **Ubiquitous language (lenguaje ubicuo)**: el mismo vocabulario que usan los expertos de negocio debe usarse literalmente en el código, dentro de cada contexto delimitado.

La práctica común hoy: cada microservicio debería alinearse con un Bounded Context — no con una tabla de base de datos ni con una capacidad técnica arbitraria.

## CONCEPTO

# CQRS: separar el camino de escritura del de lectura

**CQRS (Command Query Responsibility Segregation)**, un término acuñado por Greg Young a partir del principio de Bertrand Meyer de **separación comando-consulta**, propone usar modelos distintos para escribir datos (comandos) y para leerlos (consultas), en vez de un único modelo que sirva para ambas cosas.

- **Lado de comandos:** optimizado para validar reglas de negocio y aplicar cambios de forma consistente.
- **Lado de consultas:** optimizado para leer rápido, a menudo con una estructura de datos desnormalizada y distinta a la de escritura.

En un sistema simple, un solo modelo CRUD basta. CQRS se justifica cuando los patrones de lectura y escritura son tan distintos entre sí que un modelo único se vuelve un compromiso pobre para ambos.

## CONCEPTO

# Event Sourcing: guardar el historial completo, no solo el estado final

**Event Sourcing** es un patrón en el que el sistema no guarda el estado actual de un dato, sino la secuencia completa de eventos que lo llevaron a ese estado — el estado actual se reconstruye, en cualquier momento, reproduciendo esos eventos en orden.

Ejemplo: en vez de guardar `saldo: 850`, el sistema guarda la secuencia `DepósitoRealizado(1000)`, `RetiroRealizado(200)`, `RetiroRealizado(50)` — el saldo actual (850) se calcula sumando esos eventos, y el historial completo queda disponible para auditoría.

Este patrón se documenta ampliamente, entre otros, en los trabajos de Martin Fowler y del propio Greg Young.

CONTRAEJEMPLO

# CRUD tradicional vs. Event Sourcing

|                               | CRUD tradicional                                      | Event Sourcing                                         |
|-------------------------------|-------------------------------------------------------|--------------------------------------------------------|
| Qué se guarda                 | Solo el estado actual, se sobrescribe con cada cambio | La secuencia completa de eventos, nunca se sobrescribe |
| Historial de cambios          | Se pierde, a menos que se implemente auditoría aparte | Existe por diseño — es literalmente lo que se guarda   |
| Reconstruir un estado pasado  | No es posible sin un mecanismo adicional              | Se reproducen los eventos hasta ese punto              |
| Complejidad de implementación | Baja, bien conocida                                   | Alta — requiere infraestructura propia (event store)   |

Event Sourcing suele combinarse con CQRS (s12): los eventos alimentan el lado de escritura, y proyecciones derivadas de esos eventos alimentan el lado de lectura.

## CONCEPTO

# API Gateway: la puerta única hacia cientos de microservicios

Un API Gateway es un componente que se sitúa entre los clientes externos y el conjunto de microservicios internos, exponiendo un único punto de entrada — resolviendo, a escala de todo el sistema, varias de las prácticas vistas en la sesión anterior:

- **Enrutamiento:** dirige cada solicitud entrante al microservicio correcto.
- **Autenticación centralizada:** valida credenciales una sola vez, en vez de que cada microservicio lo haga por separado.
- **Rate limiting:** aplica límites de tasa (Semana 15, sesión anterior) de forma centralizada.
- **Agregación:** puede combinar respuestas de varios microservicios en una sola respuesta al cliente.

CASO · DE VUELTA A NETFLIX

## **Zuul: el API Gateway que Netflix construyó y abrió al público**

Con cientos de microservicios y millones de dispositivos cliente distintos (TVs, consolas, navegadores, apps móviles) conectándose a su plataforma, Netflix necesitaba exactamente el componente descrito en s15. En 2013 anunció públicamente **Zuul**, su propio API Gateway, liberado como proyecto de código abierto.

Zuul se encarga en Netflix del enrutamiento dinámico, la autenticación y la resiliencia ante fallos de los microservicios internos — la misma arquitectura descompuesta que vimos nacer en s05, ahora con una puerta de entrada única y documentada.

CONCEPTO

# Serverless y Function as a Service (FaaS)

Serverless es un modelo de ejecución en la nube donde el proveedor gestiona por completo la infraestructura del servidor, y el desarrollador solo escribe y despliega funciones individuales. Su implementación más común es **FaaS (Function as a Service)**: cada función se ejecuta bajo demanda, en respuesta a un evento (una solicitud HTTP, un archivo subido, un mensaje en una cola).

| Característica      | Qué significa                                                                                       |
|---------------------|-----------------------------------------------------------------------------------------------------|
| Escalado automático | De cero a miles de ejecuciones simultáneas, sin configuración manual.                               |
| Pago por uso real   | Se cobra por tiempo de ejecución, no por servidor reservado.                                        |
| Cold start          | La primera invocación tras un período inactivo puede tardar más, mientras se inicializa el entorno. |

TENSIÓN

# Microservicios en contenedores vs. Serverless

Con FaaS ya definido, la pregunta natural es cuándo preferirlo sobre microservicios tradicionales desplegados en contenedores (como los de Netflix, s05).

|                                 | Microservicios en contenedores                          | Serverless / FaaS                             |
|---------------------------------|---------------------------------------------------------|-----------------------------------------------|
| Control de infraestructura      | Alto — el equipo decide runtime, recursos, orquestación | Bajo — el proveedor decide casi todo          |
| Carga sostenida y predecible    | Más eficiente en costo                                  | Puede salir más caro a alto volumen constante |
| Carga esporádica o impredecible | Recursos ociosos cuando no hay tráfico                  | Ideal — se paga solo por lo que se ejecuta    |
| Latencia crítica                | Sin cold starts                                         | El cold start puede ser un problema           |

CONCEPTO EN DISPUTA

## "Serverless" no significa que no haya servidores

El nombre "serverless" genera una confusión frecuente: **los servidores siguen existiendo** — el proveedor cloud los sigue operando físicamente. Lo que desaparece no es el servidor, es la responsabilidad del equipo de desarrollo de aprovisionarlo, parchearlo y escalarlo manualmente.

Es una crítica terminológica real y frecuente en la industria — no invalida el modelo, pero sí es importante para no venderlo (ni entenderlo) mal: sigue siendo infraestructura real, solo que gestionada por otro equipo.

## INTERACCIÓN

# Diseñen la frontera de sus microservicios

**En equipos, sobre su proyecto CodeF@ctory:**

1. Identifiquen al menos dos posibles Bounded Contexts (s11) en su dominio de negocio. ¿Qué término usa cada uno que podría significar algo distinto en el otro?
2. ¿Alguna funcionalidad de su proyecto se beneficiaría de CQRS (s12) porque sus patrones de lectura y escritura son muy distintos?
3. ¿Qué parte de su sistema desplegarían como función serverless en vez de como servicio siempre activo, y por qué (s18)?

5 minutos · respondan en voz alta o en el chat

## SÍNTESIS

# Ideas clave de hoy

1. El incidente de 2008 y la migración de siete años de Netflix (2009-2016) son el caso real más citado de transición de un monolito a una arquitectura **cloud native** (contenedores, orquestación dinámica, microservicios).
2. **DDD** (Bounded Context, lenguaje ubicuo) es la herramienta principal para decidir dónde trazar el límite entre un microservicio y otro.
3. **CQRS** separa el modelo de escritura del de lectura; **Event Sourcing** guarda la secuencia de eventos en vez del estado final — ambos suelen combinarse.
4. El **API Gateway** (como Zuul, de Netflix) centraliza enrutamiento, autenticación y rate limiting frente a un conjunto grande de microservicios.
5. **Serverless/FaaS** traslada la gestión de infraestructura al proveedor; sigue habiendo servidores, solo que ya no son responsabilidad del equipo de desarrollo.

SÍNTESIS · CIERRE DE LA UNIDAD 4

## Cuatro sesiones, una sola unidad: Los Servicios Web en la Arquitectura

Con esta sesión cerramos la Unidad 4 completa, cuatro sesiones a lo largo de las semanas 14 y 15:

- **Esquemas para el Uso de Servicios Web y Arquitecturas SOA** (Semana 14) — los principios de servicios independientes y débilmente acoplados, con el mandato de Jeff Bezos en Amazon (2002).
- **Estándares y Protocolos** (Semana 14) — de SOAP y WSDL a REST, HATEOAS, HAL y GraphQL, con el caso de Facebook (2012).
- **Web APIs e Implementación de Servicios Web** (Semana 15) — versionado, paginación, autenticación e idempotencia, con Stripe frente a la ruptura de la API de Twitter (2012).
- **Cloud Native, Microservicios y Serverless** (hoy) — cómo esos mismos servicios se organizan y despliegan a la escala de un sistema completo, con la migración de Netflix.

**Toda la Unidad 4 respondió una sola pregunta: una vez que un sistema decide comunicarse mediante servicios (Unidades 1-3), ¿con qué estándares, prácticas y arquitecturas concretas se construyen, exponen y escalan esos servicios en el mundo real?**

CIERRE

## Para la próxima semana

- Repasar los cuatro patrones de esta sesión (Cloud Native, DDD, CQRS/Event Sourcing, API Gateway, Serverless) y ubicar cuáles ya usa, o podría usar, su proyecto de CodeF@ctory.
- Preparar la Entrega 2 (integración de componentes y framework), que se apoya directamente en las Web APIs y la arquitectura de servicios vistas en toda la Unidad 4.

**La Semana 16 abre la Unidad 5 — Aspectos de Seguridad en Arquitecturas de Software — con los principios de arquitectura de software segura y la importancia de la seguridad en las APIs.**

